

UCS645

PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 5:

Submitted by: Ritwik Gupta (102303096)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Question 1: DAXPY Loop Operation

1. Objective

To implement the Double-precision $A \cdot X + Y$ (DAXPY) operation using MPI and measure the speedup compared to a sequential uniprocessor implementation.

2. Problem Description

- **Vectors:** X and Y of size $N=216$ (65,536).
- **Scalar:** a (Double precision).
- **Operation:** $X[i] = a \cdot X[i] + Y[i]$.
- **Goal:** Distribute the workload across MPI ranks and compare execution time with a single-core run.

3. Methodology

- Initialize vectors X and Y on the root process (Rank 0).
- Calculate the chunk size (N/size) and scatter the data to all processes using `MPI_Scatter`.
- Each process performs the DAXPY calculation on its local chunk.
- Gather results back to the root using `MPI_Gather`.
- Measure execution time using `MPI_Wtime()`.

4. Results

Processors	Execution Time (s)	Speedup
1	0.001093	1
2	0.000802	1.363
4	0.000548	1.994
8	0.003636	0.301

5. Discussion

- For $N=216$, the computation is relatively light. The overhead of `MPI_Scatter` and `MPI_Gather` might outweigh the parallel gain on small clusters.
- Speedup is expected to increase as the vector size grows, overcoming the fixed communication latency.

6. Conclusion

The DAXPY operation was successfully parallelized. The efficiency depends heavily on the ratio of computation to communication overhead.

Question 2: The Broadcast Race (Linear vs. Tree)

1. Objective

To compare the efficiency of a manual point-to-point "Linear" broadcast versus the optimized "Tree-based" MPI_Bcast collective operation.

2. Problem Description

- Data: A massive array of 10 million doubles (~80 MB).
- Linear (MyBcast): Rank 0 uses a for loop to MPI_Send to every other rank.
- Collective (MPI_Bcast): Uses the built-in, optimized function.

3. Methodology

- Allocate 10M doubles on the heap.
- Implement MyBcast using a simple loop of MPI_Send and corresponding MPI_Recv.
- Measure time for MyBcast, then reset and measure MPI_Bcast.
- Test across 2, 4, 8, and 16 processes.

4. Results

Processes	MyBcast Time (s)	MPI_Bcast Time (s)
1	0.000	0.000
2	0.051	0.029
4	0.144	0.017
8	0.451	0.115

5. Discussion

- **Linear Scaling:** MyBcast scales linearly ($O(P)$) because Rank 0 must send the data $P-1$ times sequentially.
- **Tree Scaling:** MPI_Bcast uses a logarithmic approach ($O(\log P)$). Rank 0 sends to Rank 1, then both send to Ranks 2 and 3, doubling the participating senders each step.
- The performance gap widens significantly as the number of processes increases.

6. Conclusion

Built-in MPI collective operations are superior for large data transfers due to their logarithmic complexity and hardware-aware optimizations.

Question 3: Distributed Dot Product & Amdahl's Law

1. Objective

To calculate the dot product of massive vectors (500 million elements) and analyze parallel efficiency and Amdahl's Law.

2. Problem Description

- **Operation:** $\sum(A[i] \cdot B[i])$.
- **Memory Management:** Elements are generated locally on each rank to avoid memory overflow on Rank 0.
- **Reduction:** Local sums are aggregated using `MPI_Reduce` with `MPI_SUM`.

3. Methodology

- Rank 0 broadcasts a multiplier.
- Each rank calculates its local range and populates local arrays.
- Local dot products are computed in parallel.
- `MPI_Reduce` collects the final sum.

4. Results

Processes	Execution Time (s)	Speedup	Efficiency
1	30.133	1	1
2	17.099	1.762	0.881
4	15.979	1.886	0.472
8	12.350	2.440	0.305

5. Discussion

- Perfect Linear Speedup: Rarely achieved due to the "Sequential Fraction" (initialization, broadcast, and reduction).
- Amdahl's Law: The maximum speedup is limited by the time spent in communication and non-parallelizable code.
- Overheads: Network latency and memory bandwidth limits on the host machine.

6. Conclusion

Parallel dot product effectively handles datasets that exceed the memory/processing power of a single node.

Question 4: Dynamic Prime Number Search

1. Objective

To implement a dynamic load-balancing model using the Master-Slave paradigm to identify prime numbers within a specified range, utilizing `MPI_Recv` and `MPI_Send` for on-demand task distribution.

2. Problem Description

The search for prime numbers involves variable computational intensity (testing larger numbers often takes longer). A static distribution would lead to idle processes.

- **The Master:** Iterates from 2 to a maximum value, distributing integers to slaves as they become available.
- **The Slaves:** Request a number from the master, perform the primality test, and return a status code (positive for prime, negative for non-prime).

3. Methodology

- **Initial Handshake:** Slaves send a "zero" message to the Master to signal they are ready for their first task.
- **Dynamic Allocation:** The Master uses `MPI_Recv` with `MPI_ANY_SOURCE` to catch results from whichever slave finishes first.
- **Task Distribution:** Upon receiving a result or a ready signal, the Master sends the next integer to that specific slave.
- **Termination:** Once the maximum value is reached, the Master sends a termination signal (e.g., `-1`) to all slaves to safely shut down the communicator.

4. Results

- Search Range: 2 to 10,000

Processes	Execution Time (s)	Master	Slave	Speedup
2	0.046	1	1	1
4	0.041	1	3	1.121
8	0.143	1	7	0.319

5. Discussion

This approach implements Dynamic Load Balancing. In a parallel system, if tasks have unequal "weights" (computational costs), a static split causes a "bottleneck" where the entire program waits for the slowest process. By using `MPI_ANY_SOURCE`, we ensure that as soon as a core is free, it receives a new number to test, maximizing throughput.

6. Conclusion

The Master-Slave model successfully balanced the computational load. The use of asynchronous-style communication via `MPI_Recv` is highly effective for irregular workloads like primality testing.

Question 5: Distributed Perfect Number Discovery

1. Objective

To identify "Perfect Numbers" (where the sum of proper divisors equals the number itself) using a master-slave architecture to manage distributed mathematical verification.

2. Problem Description

- **Mathematical Criteria:** A number n is perfect if $\sigma(n) - n = n$, where $\sigma(n)$ is the sum of all divisors.
- **Communication Logic:**
 - Slaves return the number itself if it is perfect.
 - Slaves return the negative of the number if it is not perfect.
 - The Master tracks which numbers are validated and outputs the results.

3. Methodology

1. **Task Request:** Each slave starts by sending a 0 to the Master.
2. **Divisor Summation:** Slaves implement an $O(\sqrt{n})$ algorithm to find and sum all proper divisors of the received integer.
3. **Feedback Loop:** The Master listens for replies. If a slave returns a positive value, the Master logs it as a "Perfect Number".
4. **Scaling:** The system scales the search range across all available MPI ranks.

4. Results

- Search Limit: 10,000
- Perfect Numbers Identified: 6, 28, 496, 8128

Processes	Execution Time (s)	Speedup	Efficiency
2	0.055	1	0.5
4	0.060	0.92	0.23
8	0.131	0.4	0.05

5. Discussion

Unlike vector addition, finding perfect numbers becomes exponentially harder as n increases. This specific problem benefits from the Master-Slave model because the time taken to sum divisors for 100 is significantly less than for 10,000. The dynamic nature of the assignment prevents the "straggler" problem common in distributed systems.

6. Conclusion

The MPI implementation effectively identified all perfect numbers within the range. The project demonstrates that for non-uniform computational tasks, a centralized manager (Master) distributing work to a pool of workers (Slaves) is an optimal design pattern for parallel computing.